

Reviewer Pack — Minimal Local Indeterminacy in Algebraic Topology and Commun...

Entry 02fc1da3ed40 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:54:15 UTC

Minimal Local Indeterminacy in Algebraic Topology and Communication Matrix Rank

Entry ID: 02fc1da3ed40 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:54:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Communication Complexity

Statement:

The minimal local indeterminacy of a homotopy type, measured by the number of generators of its fundamental group minus the rank of its abelianization, is linearly correlated with the rank of the communication matrix for the problem associated with the homotopy type.

Rationale (proposer's reasoning):

Algebraic topology provides tools to study the connectivity and shape of spaces. The local indeterminacy captures the complexity in terms of fundamental groups, while communication complexity measures the minimum number of bits required to describe a protocol between parties. This conjecture suggests that these seemingly unrelated fields may be connected through some underlying topological structure.

Taxonomy category: ALGEBRAIC_TOPOLOGY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a59232f6d9e29e2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between minimal local indeterminacy and communication matrix rank from 30 random seeds is greater than or equal to 0.5, with no seed producing a Pearson correlation coefficient less than 0.3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local indeterminacy" AND algebraic topology AND communication complexity" - "homotopy type" AND "fundamental group" AND communication matrix rank" - "abelianization" AND algebraic topology INTEGRATED WITH communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                continue
            for j in range(n):
                A[i][j] /= pivot
            for k in range(m):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_rank(A):
        A_rref = gaussian_elimination([row[:] for row in A])
        rank = 0
        for row in A_rref:
            if any(row):
                rank += 1
        return rank

    def fundamental_group_rank(n):
        # Simulate the rank of the fundamental group (example: n generators)
```

```

    return n

def abelianization_rank(n):
    # Simulate the rank of the abelianization (example: n-1 generators)
    return n - 1

def communication_matrix_rank(n):
    # Simulate a random communication matrix and compute its rank
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    return matrix_rank(A)

n_values = [5, 10, 15, 20, 30, 40]
local_indeterminacies = []
comm_matrix_ranks = []

for n in n_values:
    f_group_rank = fundamental_group_rank(n)
    abelian_rank = abelianization_rank(n)
    local_indeterminacy = f_group_rank - abelian_rank
    comm_matrix_rank_val = communication_matrix_rank(n)

    local_indeterminacies.append(local_indeterminacy)
    comm_matrix_ranks.append(comm_matrix_rank_val)

if not local_indeterminacies or not comm_matrix_ranks:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_data"
    }

n = len(local_indeterminacies)
mean_local_indeterminacy = sum(local_indeterminacies) / n
mean_comm_matrix_rank = sum(comm_matrix_ranks) / n

covariance = sum((local_indeterminacies[i] - mean_local_indeterminacy) * (comm_matrix_ranks[i] - mean_comm_matrix_rank) for i in range(n))
variance_local_indeterminacy = sum((local_indeterminacies[i] - mean_local_indeterminacy) ** 2 for i in range(n))
variance_comm_matrix_rank = sum((comm_matrix_ranks[i] - mean_comm_matrix_rank) ** 2 for i in range(n))

pearson_corr_coeff = covariance / (variance_local_indeterminacy * variance_comm_matrix_rank)

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_corr_coeff,
    "instances_tested": n,
    "n_max": max(n_values),
    "conjecture_holds": pearson_corr_coeff >= 0.5 and all(pearson_corr_coeff >= 0.3 for _ in range(n)),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

```

```

results = []

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")

mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
std_metric_value = (sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results)) ** 0.5
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results) and all(result["metric_value"] < 1 for result in results):
    print("RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed=1")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f926863d.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f926863d.py", line 67, in run_trial
    comm_matrix_rank_val = communication_matrix_rank(n)
                          ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f926863d.py", line 57, in communication_matrix_rank
    return matrix_rank(A)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f926863d.py", line 39, in matrix_rank
    A_rref = gaussian_elimination([row[:] for row in A])
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f926863d.py", line 25, in gaussian_elimination
    A[i], A[max_row] = A[max_row], A[i]
                        ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required calculations for the Pearson correlation coefficient. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13096
2	propose	ollama_remote	glm4:latest	0	0	14542
3	preregistration	ollama_remote	glm4:latest	0	0	17305
4	novelty	ollama_remote	glm4:latest	0	0	14223
5	novelty	ollama_remote	glm4:latest	0	0	9222
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17906
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17330
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13373
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14632
10	judge	ollama_remote	glm4:latest	0	0	8657

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140287 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/02fc1da3ed40.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/02fc1da3ed40.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/02fc1da3ed40.tar.gz* (if generated)